COMPX234-26A - Assignment 1

March 17, 2026

Total Marks: 10 (5 marks for code, 5 marks for demonstration)
Due: 23:59, Friday, 10th April 2026

Your Ethical Responsibilities

- Students found plagiarising will be reported to the disciplinary committee (see university guidelines https://www.waikato.ac.nz/students/academic-integrity/student-information/plagiarism)
- Assignments may be checked using anti-plagiarism tools.

1 The Problem

This assignment is an example of the **Producer-Consumer** problem, and is designed to **complement** the Processes you have learnt in the lectures.

In our system, we have several **machines** (`M`) that occasionally need to print some text. We also have a smaller number of **printers** (`N`) that can print text given to them. In this system, there are more machines than printers (`M > N`).

To enable printing, the system uses a **print queue**. When a machine has something to print, it creates a print request for the document and inserts that request into the print queue. When a printer becomes free, it retrieves the document at the head of the queue, prints it, and removes it from the queue.

Because resources in a real system are finite, the queue is also limited. In this assignment, the queue size is equal to the number of printers in the system. This creates a synchronization problem: if the queue is already full and a machine creates a new print request, the request could be appended to the end of the queue and cause the head element to be lost. This overwriting behaviour is undesirable.

Your job is to use **threads** and **synchronization** in C# to prevent this from happening.

2 Starter Code

You have been given the following C# files:

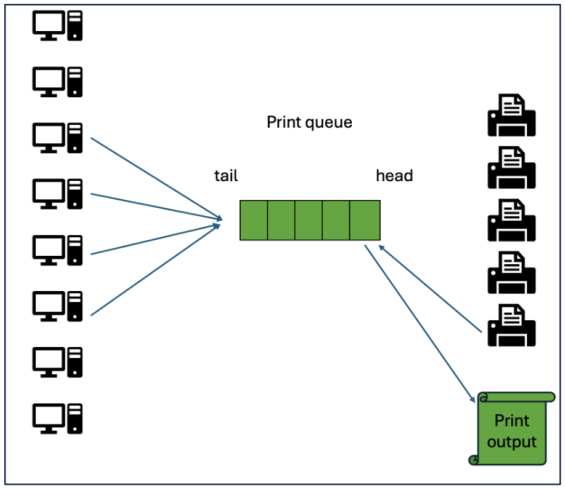

Figure 1: Visual representation of the system

- `Assignment1.cs`
- `printDoc.cs`
- `printList.cs`

2.1 File roles

- `printDoc.cs` defines the print request object.
- `printList.cs` defines the queue structure and provides queue operations such as inserting and printing requests.
- `Assignment1.cs` is the file students should complete.

You should keep the provided structure as much as possible. The only file you need to change to complete the assignment is the main File, `Assigment1.cs`. The other files have been provided for your understanding.

3 Tasks

3.1 Task 1

Using the starter code in `Assignment1.cs`, implement the following behaviour using **threads in C#**. You must use the Thread class for this assignment. Do not use Task.Run.

1. Machines sleep for some time, wake up, and then send a print request.
2. Printers also sleep for some time. When they wake up, they print the document at the head of the queue.

```
Machine 21 Sent a print request
Machine 22 Sent a print request
Inserted a request in the queue from 22
!!!!!!Attention: Overwrite!!!!!!
Number of requests in the queue 5
Machine 11 Sent a print request
Inserted a request in the queue from 11
!!!!!!Attention: Overwrite!!!!!!
```

Figure 2: Sample output after Task1

The starter code already contains methods that help you implement this behaviour. In Task 1 you will use these methods to obtain the above functionality. When your program runs, the output should show:

- machines sending print requests,
- printers printing requests, and
- queue activity.

When this is done, you should see overwrite behaviour when the queue becomes full as shown in Figure 2.

In `Assignment1.cs` file you will find `printerThread` and `machineThread` classes for printers and machines respectively. The starting point for the printer threads is the `SetPrinter` function and for machine threads is the `SetMachine` function. You can create a printer thread and add it to the List like so,

```
pThreads.Add(new Thread(new printerThread(this, i).SetPrinter));
```

Machine threads can be created likewise.

3.2 Task 2

Design and implement a synchronization solution using the techniques discussed in lectures to prevent overwriting.

Your solution must:

1. Control the machines' access to the print queue so that no overwriting occurs.

   If there are already 5 messages in the queue, no machine should be allowed to send another print request until a printer removes one and space becomes available.

2. Control access to the queue so that **no two devices** (machines and/or printers) access the queue at the same time.

```
Machine 5 Checking availability
Machine 5 will proceed
Machine 5 Sending a print request
Inserted a request in the queue from 5
Number of requests in the queue 4
Machine 5 Releasing binary semaphore
Machine 6 Checking availability
Machine 6 will proceed
Machine 6 Sending a print request
Inserted a request in the queue from 6
Number of requests in the queue 5
Machine 6 Releasing binary semaphore
Printer ID:0 : now available
:::::
Printer 0 Printing the request from Machine ID: 1 My name is machine 1
:::::
Printer ID:0 Releasing semaphores
Printer ID:3 : now available
Printer ID:4 : now available
:::::
Printer 3 Printing the request from Machine ID: 2 My name is machine 2
:::::
Printer ID:3 Releasing semaphores
:::::
Printer 4 Printing the request from Machine ID: 3 My name is machine 3
:::::
Printer ID:4 Releasing semaphores
Machine 2 Checking availability
Machine 2 will proceed
Machine 2 Sending a print request
```

Figure 3: Sample output after Task2

4 Compilation and Running

This assignment is intended to compile with `mcs` and run with `mono` on Linux (or any Bash terminal where those tools are installed). You can use a .NET environment and compile it on a Windows machine to develop and test but the marking will take place on a Linux environment. Test your code on the Linux machines in R.G.07 before submission.

4.1 Compile

```
mcs Assignment1.cs printDoc.cs printList.cs
```

4.2 Run

```
mono Assignment1.exe
```

5 Submission

- We are not using gitlab, but it is suggested you create your own repository following best code practices to save and track your work for this assignment, or use Onedrive on the lab computers.
- This assignment requires verification of your assignment in the COMPX234 Labs held. As part of your assessment you will be asked to explain the logic, functionality, parts of your code as well as the choices you made to show understanding.

6 Extension Requests

Extension requests should be made at least two days before the due date using the extension form on moodle. You will need supporting evidence such as a doctor's certificate to attach with the extension request.